

MetaCognition: 可验证的跨模型认知架构

产品需求文档 (PRD) + 技术规范 (Spec) + 业务逻辑设计
为 Claude Code 构建有状态的、跨会话的、多角色专家会诊系统。
版本: 2.0 | 日期: 2026-05-20 | 状态: 设计定稿

目录

- 产品概述
 - 1.1 愿景与使命
 - 1.2 核心概念
 - 1.3 目标用户
- 应用场景
 - 2.1 场景一: 复杂架构重构
 - 2.2 场景二: 诡异 Bug 排查
 - 2.3 场景三: 安全敏感变更
 - 2.4 场景四: TDD 测试驱动开发
 - 2.5 场景五: 性能优化
 - 2.6 场景六: 跨团队协作
 - 2.7 场景七: 新人 onboarding
- 业务逻辑
 - 3.1 双态知识生命周期
 - 3.2 三级复杂度自适应路由
 - 3.3 六重决策门
 - 3.4 外脑会诊流程
 - 3.5 两阶段验证闭环
 - 3.6 知识固化与晋升
 - 3.7 熔断与配额管理
 - 3.8 知识衰减与归档
- 架构设计
 - 4.1 三层认知架构
 - 4.2 认知层 (Karpathy Wiki)
 - 4.3 物理层 (CodeGraph)
 - 4.4 推理层 (Expert Brain MCP)
- 功能规格
 - 5.1 MetaCognition Skills
 - 5.2 Expert Brain MCP Server
 - 5.3 与 Superpowers 融合
 - 5.4 TDD 深度适配
- 配置与部署
 - 6.1 完整配置示例
 - 6.2 Claude Desktop 配置
 - 6.3 启动时校验
- 实施路线图
- 附录
 - 8.1 术语表

1. 产品概述

1.1 愿景与使命

愿景：让 AI 辅助编程从” 单次会话的聪明” 进化为” 跨会话的睿智”——不仅解决当前问题，更记住为什么解决、如何避免重蹈覆辙、如何在团队间传承经验。

使命：为 Claude Code 构建一套**认知增强架构**，解决当前 AI 辅助编程的三大核心痛点：

1. **瞬时经验流失：**模型排查的诡异 Bug、发现的环境陷阱，在会话关闭后立即消失，下次遇到同类问题时从零开始。
2. **盲目猜测与无效重试：**遭遇超出模型全局知识边界的深度技术屏障时，模型倾向于在当前受限上下文内连续盲目重试，既消耗 Token 又破坏代码树结构。
3. **规则库过度膨胀：**若将所有偶发踩坑经验直接写入全局静态约束文件，会导致主智能体在处理微小任务时背负过于沉重的负向检索包袱，进而产生幻觉或拒绝执行。

1.2 核心概念

概念一：双态知识生命周期 (Dual-State Knowledge Lifecycle) 知识在 MetaCognition 中经历两种状态：

- **草稿态 (Draft)：**高容错、轻量级捕获。任何被成功解决的诡异错误、非显而易见的陷阱，都可以被快速记录为 draft insight。不要求完美，不要求立即验证，只要求” 比遗忘好”。
- **发布态 (Live)：**经严格验证后固化的防御红线。只有满足 Hit Count ≥ 5 、通过沙盒验证、无冲突、3 个月内有命中的洞察，才能晋升为 Live Constraint。Live 约束直接回写到 CLAUDE.md 或持久化到 SQLite 知识引擎。

概念二：三级复杂度自适应路由 (3-Tier Complexity Adaptive Routing) 不是所有任务都需要完整的认知守护。系统根据变更规模自动选择触发级别：

- **Micro** («5 行, 1 个文件)：仅 Librarian 自动预检，零感知通过。
- **Light** («50 行, ≤ 3 个文件)：选择性触发 Security + QA Gate。
- **Full** (新模块/依赖/API 变更)：强制全量六重决策门 + Deliberation。

概念三：六重决策门 (Six Decision Gates) 在关键操作前强制触发的认知检查点：

1. **Anti-Pattern Gate：**每次 tool call 前自动拦截，检查负向约束。
2. **Architecture Gate：**新增模块、重大重构、跨服务调用、引入依赖。
3. **Security Gate：**用户输入处理、文件上传、权限变更、外部 API 调用。
4. **Debug Gate：**同一错误修复 ≥ 2 次失败、未知异常、性能衰退。
5. **QA Gate：**新功能开发、复杂边界条件、历史 bug 区域。
6. **Performance Gate：**DB 查询修改、循环嵌套、缓存策略调整。

概念四：三层认知-物理-推理架构 (Cognitive-Physical-Reasoning Trinity)

- **认知层 (Cognitive Layer)：**基于 Karpathy LLM Wiki 的状态化知识编译。项目拥有一个由 LLM 自动维护的、交叉索引的专属知识维基。
- **物理层 (Physical Layer)：**基于 CodeGraph 的 AST 级代码拓扑感知。系统拥有类似现代 IDE 的结构感知与静态分析能力。
- **推理层 (Reasoning Layer)：**基于 Expert Brain MCP 的多角色外脑会诊。主脑在执行任务时，遇难题呼叫外脑破局。

1.3 目标用户

用户角色	核心痛点	MetaCognition 价值
资深架构师	反复向 AI 解释同一架构原则，AI 仍犯错	架构约束自动固化，跨会话稳定遵循
全栈开发者	调试诡异环境问题时 AI 盲目重试，破坏代码	Debug Gate 强制外脑会诊，避免无效重试
安全工程师	AI 生成的代码反复出现同一安全漏洞	Security Gate + 负向约束固化，漏洞零复现
TDD 实践者	AI 写的测试脆弱、重构时测试雪崩	TDD 全阶段认知守护，测试构造经验沉淀
团队 Lead	新人反复踩老人踩过的坑，知识无法传承	团队级 Wiki 知识库，经验自动晋升与共享
性能工程师	AI 优化建议未经验证，引入退化	Performance Gate + Sandbox 验证，证据高于宣称

2. 应用场景

2.1 场景一：复杂架构重构

场景描述 你正在重构一个核心模块 `parser.py` 中的 `parse_config` 函数。这个函数被 3 个上游模块调用，间接影响 8 个测试文件。你让 Claude Code 执行重构，但它不知道这个函数的历史脆弱性——3 个月前的一次重构曾导致 8 个测试文件中的 5 个失败，原因是隐式依赖了副作用。

传统工作流的问题

- 1. 经验流失: 3 个月前的重构教训只存在于某个已关闭的会话中，Claude 完全不记得。
- 2. 盲目执行: Claude 直接修改 `parse_config`，没有检查调用链，导致测试雪崩。
- 3. 无效重试: 测试失败后，Claude 尝试 4-5 种不同的修复方案，每种都引入新的副作用，代码树逐渐混乱。
- 4. 上下文污染: 如果你把”parser.py 重构要小心” 写进 CLAUDE.md，这条约束会在每次会话加载，即使你在修改完全不相关的 CSS 文件时也会背负这条规则。

MetaCognition 工作流 Step 1: Session Start — Librarian 预检

Claude Code 启动时，Librarian 自动检索 `negative_constraints`。发现一条 high-severity 约束：

```
## Live Constraint: parser_refactoring_fragility
** 触发 **: 修改 src/parser.py:parse_config
** 历史 **: 3 个月前重构导致 5/8 测试失败 ([[draft_20260315_parser_refactor]])
** 根因 **: parse_config 的隐式副作用被调用者依赖
** 防范 **: 重构前必须先分析调用链，为所有调用者补充测试
** 验证 **: 必须通过 Sandbox 运行全部 8 个关联测试
```

Step 2: Architecture Gate — 外脑会诊

当你要求重构 `parse_config` 时，系统检测到 Full 复杂度（跨模块变更），强制触发 Architecture Gate：

- 1. CodeGraph 物理感知: 分析 `parse_config` 的调用链，生成 Impact Graph:
 - 直接调用者: 3 个函数 (`main.py:load_config`, `cli.py:init`, `api.py:parse_request`)

- 间接影响: 8 个测试文件
- 测试覆盖: 2/3 调用者有直接测试, 1 个 (`api.py:parse_request`) 无测试
- 风险评分: 0.7 (高, 因为未测试调用者在关键路径)

2. **Architect 外脑会诊:** Claude 调用 `expert_brain/consult`, 注入 Impact Graph + 历史约束:

Architect 返回提案:

```
## 重构方案: parse_config 接口隔离

** 阶段 1 **: 引入 ConfigParser 接口
- 创建 `src/interfaces/config_parser.py`
- 将 `parse_config` 的副作用提取到 `ConfigParserSideEffect`

** 阶段 2 **: 逐步迁移调用者
- 为 `api.py:parse_request` 补充单元测试 (当前无测试, 风险最高)
- 迁移 `main.py:load_config` → 使用新接口
- 迁移 `cli.py:init` → 使用新接口

** 阶段 3 **: 验证
- 运行全部 8 个关联测试
- 运行性能基准 (parse_config 是热路径)
```

3. **Deliberation 综合审议:** QA 专家审查测试策略, Performance 专家审查性能影响, Deliberation Synthesizer 综合三方意见:

- 共识: 接口隔离是正确的方向
- 张力: QA 认为应该先补测试再迁移, Architect 认为可以先迁移再补测试 (因为现有测试已覆盖 2/3 调用者)
- 推荐路径: 先迁移已测试的调用者, 同时为未测试调用者写测试, 最后迁移
- **Fallback:** 如果测试编写耗时 >30 分钟, 先标记 `api.py:parse_request` 为技术债务, 后续迭代

Step 3: 执行与验证

Claude 按推荐路径执行:

1. 引入 ConfigParser 接口
2. 迁移 `main.py:load_config` 和 `cli.py:init`
3. 为 `api.py:parse_request` 写测试
4. 迁移 `api.py:parse_request`
5. **Sandbox 验证:** 运行全部 8 个关联测试 → 全部通过
6. **Quality Audit:** 检查副作用 → 无预期外文件修改

Step 4: 经验沉淀

系统自动记录 draft insight:

```
## Draft Insight: parser_config_interface_migration
**Symptom**: parse_config 重构导致测试雪崩
**Root Cause**: 隐式副作用被调用者依赖, 且 api.py:parse_request 无测试覆盖
**Resolution**: 引入 ConfigParser 接口, 按 "已测试 → 未测试" 顺序迁移
**Hit Count**: 1
**Context**: [[draft_20260315_parser_refactor]] 的演进
```

3 个月后，当另一位开发者尝试重构 `parse_config` 时，Librarian 预检会命中这条约束，Architect 会推荐同样的接口隔离方案——经验实现了跨会话、跨开发者的复利增长。

2.2 场景二：诡异 Bug 排查

场景描述 你在 Windows 上使用 VS Code + Conda 环境，Claude Code 生成的 `bash` 脚本在终端中运行失败，提示“conda 不是内部或外部命令”。这是一个典型的环境配置陷阱，涉及 VS Code Terminal Profile、Conda Hook、PowerShell vs Git Bash 的复杂交互。

传统工作流的问题

- 1. **盲目重试**: Claude 尝试 4 种不同的 `bash` 语法 (换行符、`&&`、`heredoc`、子 `shell`)，每种都在沙盒中被拦截，浪费 20+ 轮对话。
- 2. **经验流失**: 上一次会话中，你已经告诉 Claude “VS Code 默认用 PowerShell，要改成 Git Bash”，但新会话完全遗忘。
- 3. **无效方案**: Claude 提议“用 Python 脚本替代 `bash`”，但你的 CI 环境强制要求 `bash` 脚本，方案不可行。

MetaCognition 工作流 Step 1: Debug Gate — 2 次失败后自动触发

Claude 第 1 次尝试: 使用换行符连接命令 → 沙盒权限提示失败
Claude 第 2 次尝试: 使用 `heredoc` → 沙盒不支持 → **Debug Gate 自动触发**

Step 2: Debugger 外脑会诊

Claude 调用 `expert_brain/consult` (`expert=debugger`, `problem_type="bash_sandbox_permission"`)，注入错误日志 + 尝试历史 + 环境上下文 (Windows + VS Code + Conda)。

Debugger (DeepSeek-R1) 深度推理:

```
## 根因分析

** 表层 **: bash 命令在沙盒中执行失败
** 深层 **:
1. VS Code 默认 Terminal Profile 为 PowerShell
2. PowerShell 未执行 Conda Hook (conda init powershell 未运行)
3. 因此 `conda activate` 在 PowerShell 中不可用
4. 但 bash 脚本假设 `conda` 命令可用

** 验证假设 **:
- 检查 VS Code settings.json 的 terminal.integrated.defaultProfile.windows
- 检查是否运行过 `conda init powershell` 或 `conda init bash`

** 修复路径 **:
方案 A: 修改 VS Code 默认 Terminal Profile 为 Git Bash (推荐, 一劳永逸)
方案 B: 在 bash 脚本中显式指定 conda 路径 (脆弱, 路径因环境而异)
方案 C: 使用 Python 脚本替代 bash (不可行, CI 强制要求 bash)
```

Step 3: 验证与执行

Claude 在 Sandbox 中验证方案 A:

- 1. 修改 `.vscode/settings.json`: `"terminal.integrated.defaultProfile.windows": "Git Bash"`

- 2. 运行 bash 脚本 → conda activate 成功
- 3. 验证通过

Step 4: 经验沉淀 — 联动更新 Wiki

系统自动调用 record_draft_insight, 触发 Wiki Compiler 联动更新:

```
## Wiki Node: conda_path_mismatch_in_vscode_terminal.md (新建/更新)

### Symptom
在 VS Code Terminal 中运行包含 `conda activate` 的 bash 脚本时，
提示 "conda 不是内部或外部命令"。

### Root Cause
VS Code 默认 Terminal Profile 为 PowerShell，未执行 Conda Hook。

### Resolution
修改 VS Code settings.json，将默认 Terminal Profile 改为 Git Bash:
```json
"terminal.integrated.defaultProfile.windows": "Git Bash"
```

Context

- 首次发现: 2026-05-18
- 最后命中: 2026-05-20
- Hit Count: 3
- 验证状态:  已通过沙盒验证

Backlinks

- [[python\_environment\_setup]] — 更广泛的 Python 环境配置问题
- [[vscode\_terminal\_profiles]] — VS Code Terminal 配置模式
- [[windows\_path\_issues]] — Windows 路径相关陷阱
- [[bash\_sandbox\_permission\_loop]] — 相关的 bash 沙盒权限问题

Tags

#conda #vscode #windows #terminal #environment #bash

```
** 联动更新 **:
```

```
- `python_environment_setup.md`: 追加 backlink `[[conda_path_mismatch_in_vscode_terminal]]`
```

```
- `vscode_terminal_profiles.md`: 追加 backlink `[[conda_path_mismatch_in_vscode_terminal]]`
```

```
- `index.md`: 在 " 环境配置" 分类下新增条目
```

```
- `log.md`: 追加 `2026-05-20: Ingested conda_path_mismatch (Hit: 3)`
```

```
**Step 5: 未来防护 **
```

```
1 个月后，另一位团队成员在 Windows 上遇到同样的 Conda 问题:
```

```
- Librarian 预检命中 `conda_path_mismatch_in_vscode_terminal` (high-severity)
```

```
- ** 自动拦截 **: " 检测到你在 Windows + VS Code 环境中使用 bash + conda，历史上 3 次踩坑，建议先检查 Terminal
```

```
 ↪ Profile"
```

```
- 用户确认后，Claude 直接应用已知方案，** 无需外脑会诊，零 Token 浪费 **
```

---

### ### 2.3 场景三：安全敏感变更

#### #### 场景描述

你需要为 API 端点添加文件上传功能。Claude Code 快速生成了代码，但你担心：

- 是否验证了文件类型和大小？
- 是否将上传路径与可执行目录隔离？
- 是否防止了路径遍历攻击（`../../../../../etc/passwd`）？

#### #### 传统工作流的问题

1. \*\* 事后发现 \*\*：代码已经写好了，review 时才发现缺少输入验证。
2. \*\* 重复漏洞 \*\*：3 个月前的另一个端点也出现过同样的路径遍历漏洞，但新代码完全复制了同样的错误。
3. \*\* 安全审查滞后 \*\*：安全工程师在 PR 阶段才介入，此时修复成本高。

#### #### MetaCognition 工作流

##### \*\*Step 1: Security Gate —强制触发 \*\*

Claude 检测到" 文件上传 + 用户输入处理"，自动触发 Security Gate (Full 复杂度，不可绕过)。

##### \*\*Step 2: 双路审查 —规则引擎 + LLM\*\*

1. \*\* 规则引擎预检 \*\* (<50ms):
  - 扫描代码中是否包含 `eval()`、`exec()`、`os.system()` → 未命中
  - 扫描文件路径拼接是否使用用户输入 → \*\* 命中 \*\*：`upload\_path = f"{UPLOAD\_DIR}/{filename}"`
  - 扫描是否验证文件扩展名 → \*\* 未命中 \*\*：无白名单检查
  - \*\* 风险标记 \*\*：路径遍历高危

2. \*\*Security 外脑会诊 \*\* (Claude-Opus):

**Security 返回提案 \*\*：**

```markdown

安全审查报告：文件上传端点

**** 威胁模型 **：**

- 攻击向量：路径遍历 (Path Traversal)
- 攻击场景：上传文件名为 `../../../../../app/main.py`，覆盖应用代码
- 风险等级：Critical

**** 缓解措施 **：**

1. 文件名白名单：只允许 `.jpg`、`.png`、`.pdf`
2. 路径隔离：上传目录与代码目录物理隔离（不同磁盘或 chroot）
3. 文件名随机化：使用 UUID 存储，原始文件名仅作 metadata
4. 大小限制：单文件 ≤ 10MB
5. MIME 类型验证：不仅检查扩展名，还检查文件头 magic bytes

**** 验证步骤 **：**

1. 尝试上传 `test.jpg` → 应成功
2. 尝试上传 `../../../../etc/passwd` → 应被拒绝 (400 Bad Request)
3. 尝试上传 `shell.php.jpg` → 应被 MIME 验证拒绝

Step 3: 安全方案实施与验证

Claude 按 Security 的提案修改代码:

1. 添加文件名白名单验证
2. 使用 UUID 作为存储文件名
3. 添加 MIME 类型验证 (检查文件头)
4. 添加 10MB 大小限制

Sandbox 验证:

- 正常文件上传 → 通过
- 路径遍历尝试 → 被拦截 (返回 400)
- 伪装扩展名尝试 → 被 MIME 验证拦截

Step 4: 经验沉淀

```
## Live Constraint: file_upload_path_traversal (晋升)

** 约束 **：任何文件上传端点必须满足：
1. 文件名白名单验证 (扩展名 + MIME 类型)
2. 存储文件名随机化 (UUID)
3. 上传目录与代码目录物理隔离
4. 单文件大小限制

** 历史 **：
- 首次踩坑：2026-03-10 ([[draft_20260310_upload_vuln]])
- 复现：2026-05-20 (当前场景)
- Hit Count: 5 → 自动晋升 Live Constraint

** 触发文件 **：`src/api/*upload*`, `src/routes/*file*`
** 检测正则 **：`upload.*filename|file.*path`
** 自动拦截 **：是 (Critical)
```

Step 5: 未来防护

3 个月后, 另一位开发者添加图片上传功能:

- 代码刚写完, Librarian 预检命中 `file_upload_path_traversal`
- 自动拦截: “检测到文件上传功能, 历史上 5 次路径遍历踩坑, 必须满足 4 项安全要求”
- Claude 直接应用已知安全模式, 漏洞在编码阶段即被消灭

2.4 场景四: TDD 测试驱动开发

场景描述 你正在为一个 React 组件编写测试。组件包含异步数据获取、条件渲染、用户交互。你尝试写测试, 但不确定:

- 如何正确 mock fetch 而不污染全局？
- 如何测试异步渲染的边界条件 (loading → error → success)？
- 重构时如何确保测试不脆弱（不过度依赖实现细节）？

传统工作流的问题

1. **测试构造困境**: Claude 写的测试依赖 `jest.mock('../api')`，但重构后文件路径变了，测试批量失败。
2. **异步测试脆弱**: 使用 `setTimeout` 或 `waitFor` 的魔法数字，在 CI 中间歇性失败。
3. **经验无法复用**: 上一次会话中,你已经和 Claude 一起找到了”用 MSW (Mock Service Worker) 替代 `jest.mock`”的最佳实践，但新会话完全遗忘。

MetaCognition 工作流 Step 1: Red 阶段 — 测试构造困境

你要求 Claude 为 `UserProfile` 组件写测试。Claude 第 1 次尝试：

```
// 脆弱的测试：依赖文件路径
jest.mock('../api/userApi');
```

你指出：“这个 mock 方式太脆弱，重构后路径变了就失效。”

Claude 第 2 次尝试：

```
// 仍然脆弱：使用 setTimeout
await waitFor(() => expect(screen.getByText('Loaded')).toBeInTheDocument(), { timeout: 1000 });
```

你指出：“setTimeout 魔法数字在 CI 中会失败。”

Debug Gate / QA Gate 自动触发 (2 次尝试失败)。

Step 2: QA 外脑会诊

Claude 调用 `expert_brain/consult` (`expert=qa`, `problem_type="react-async-test-construction"`)，注入尝试历史 + 组件代码。

QA (GPT-4.1) 返回测试策略：

```
## 测试策略：UserProfile 异步组件

** 问题诊断 **：
- 第 1 次失败：jest.mock 依赖文件路径，重构时脆弱
- 第 2 次失败：waitFor 超时魔法数字，CI 环境不稳定

** 推荐模式：MSW + 显式状态断言 **

```javascript
// ✅ 正确：MSW 拦截网络层，不依赖文件路径
import { server } from '../mocks/server';
import { rest } from 'msw';

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

test('renders user profile after loading', async () => {
```

```
// Arrange: 定义 API 响应
server.use(
 rest.get('/api/user/1', (req, res, ctx) =>
 res(ctx.json({ name: 'Alice', email: 'alice@example.com' })))
);

// Act: 渲染组件
render(<UserProfile userId="1" />);

// Assert: 显式状态断言, 无魔法数字
expect(screen.getByText(/loading/i)).toBeInTheDocument();

await waitForElementToBeRemoved(() => screen.queryByText(/loading/i));

expect(screen.getByText('Alice')).toBeInTheDocument();
expect(screen.getByText('alice@example.com')).toBeInTheDocument();
});
```

关键原则:

1. **MSW 替代 jest.mock**: 拦截 HTTP 层, 不依赖模块路径
2. **waitForElementToBeRemoved 替代 waitUntil**: 等待 loading 消失, 而非等待内容出现
3. **显式状态断言**: 每个测试断言 loading → success 的完整状态机
4. **无魔法数字**: 所有异步等待基于 DOM 状态, 而非固定超时

**\*\*Step 3: Green 阶段 —经验沉淀 \*\***

Claude 应用 QA 的提案, 测试通过。系统自动记录 draft insight:

```markdown

Draft Insight: react_async_test_msw_pattern

****Symptom****: React 异步组件测试脆弱, jest.mock 依赖路径, waitUntil 有魔法数字

****Root Cause****: 测试耦合实现细节 (文件路径、超时), 而非耦合行为 (DOM 状态)

****Resolution****: 使用 MSW 拦截 HTTP 层 + waitForElementToBeRemoved + 显式状态断言

****Hit Count****: 1

****Tags****: #react #testing #msw #async #tdd

Step 4: Refactor 阶段 — 性能与耦合检查

你重构 UserProfile 组件, 将其拆分为 UserProfile + UserDetails。Claude 检测到:

- 影响测试文件: 2 个 (原测试 + 新测试)
- 测试耦合度: 低 (MSW 不依赖组件内部结构)
- **Refactor Gate 未触发** (影响 <3 文件, 无性能退化)

测试全部通过, 无需修改——**MSW 模式**保障了重构安全。

Step 5: 知识晋升

3 个月后，该模式在 5 个不同组件的测试中被复用，Hit Count 达到 5:

- 自动触发 `promote_live_constraints`
- 生成 Live Constraint Node: `react_async_testing_msw_pattern.md`
- 回写到 CLAUDE.md Testing Constraints 章节

Step 6: 未来复用

1 个月后，新项目中需要写 React 异步测试:

- Librarian 检索命中 `react_async_testing_msw_pattern`
- Claude 直接应用 MSW 模式，无需再次踩坑
- 测试从第 1 次尝试即正确，零无效重试

2.5 场景五：性能优化

场景描述 你的应用在生产环境中响应变慢。监控显示数据库查询耗时增加。Claude Code 提议”加缓存”，但你担心:

- 缓存是否会导致数据不一致?
- 缓存失效策略是否合理?
- 优化后是否引入了新的性能瓶颈（如缓存序列化开销）?

传统工作流的问题

1. **盲目优化**: Claude 直接添加 Redis 缓存，但没有分析查询瓶颈，优化了不该优化的地方。
2. **无基准对比**: 优化前后没有性能基准，无法证明是否有效。
3. **副作用未知**: 缓存引入后，用户报告”数据更新后 5 分钟才生效”——缓存失效策略缺失。

MetaCognition 工作流 Step 1: Performance Gate — 强制触发

Claude 检测到”DB 查询修改 + 缓存策略变更”，自动触发 Performance Gate。

Step 2: Performance 外脑会诊

Claude 调用 `expert_brain/consult` (`expert=performance`), 注入:

- 当前查询执行计划 (EXPLAIN ANALYZE)
- 当前基准数据 (P95 响应时间: 450ms)
- 目标 SLO (P95 < 200ms)
- 资源限制 (内存 2GB, Redis 已部署)

Performance (Gemini-2.5-pro) 返回分析:

性能诊断报告

** 瓶颈定位 **:

- 查询: ``SELECT * FROM orders WHERE user_id = ? ORDER BY created_at DESC``
- 问题: 无索引, 全表扫描 + filesort
- 影响: P95 = 450ms, 该查询占请求时间的 60%

```

** 优化路径 **:
1. ** 索引优化 ** (推荐, 零副作用) :
  - 添加复合索引: `(user_id, created_at DESC)`
  - 预期提升: P95 从 450ms → 80ms
  - 验证: EXPLAIN 确认使用 index, 无 filesort

2. ** 查询重构 ** (备选) :
  - 分页查询替代全量: `LIMIT 20 OFFSET 0`
  - 预期提升: P95 → 50ms
  - 风险: 深度分页性能衰退

3. ** 缓存引入 ** (不推荐当前阶段) :
  - 理由: 瓶颈是索引缺失, 而非查询频率。缓存会掩盖根本问题。
  - 风险: 缓存一致性、失效策略复杂度
  - 建议: 索引优化后, 若 QPS > 1000/s 再考虑缓存

** 基准要求 **:
- 优化前: P95 = 450ms, P99 = 1200ms, 吞吐量 = 200 req/s
- 优化后目标: P95 < 100ms, P99 < 300ms, 吞吐量 > 500 req/s

```

Step 3: 方案实施与验证

Claude 按 Performance 的推荐, 先实施索引优化:

- 1. 添加复合索引
- 2. **Sandbox** 验证:
 - EXPLAIN 确认使用 index
 - 运行基准测试: P95 = 75ms (目标达成)
 - 运行一致性测试: 1000 次读写, 无数据不一致

Step 4: 经验沉淀

```

## Draft Insight: db_query_cache_first_anti_pattern

**Symptom**: DB 查询慢, 直接加缓存
**Root Cause**: 未分析查询计划, 缓存掩盖了索引缺失的根本问题
**Resolution**: 先 EXPLAIN ANALYZE 定位瓶颈, 优先索引/查询重构, 缓存是最后手段
**Hit Count**: 1
**Tags**: #performance #database #cache #index #optimization

```

Step 5: 未来防护

2 个月后, 另一位开发者遇到 API 响应慢:

- Performance Gate 触发
- Librarian 检索命中 db_query_cache_first_anti_pattern
- **自动提示**: “历史上遇到性能问题时, 直接加缓存是常见反模式。建议先运行 EXPLAIN ANALYZE 定位瓶颈。”
- Claude 先分析查询计划, 发现是 N+1 查询问题 → 使用 JOIN 优化 → **避免了一次无效的缓存引入**

2.6 场景六：跨团队协作

场景描述 你的团队有 5 名开发者，使用同一个代码库。开发者 A 发现了一个关于 Docker 卷挂载的陷阱，开发者 B 在 2 周后踩了同样的坑，开发者 C 在 1 个月后再遇到。

传统工作流的问题

1. **知识孤岛**：A 的经验只存在于 A 的本地 CLAUDE.md 或记忆中，B 和 C 无法访问。
2. **重复踩坑**：同一陷阱在不同开发者间反复出现，每次都要重新排查。
3. **文档腐烂**：团队 Wiki 中的” 已知问题” 页面无人维护，信息过时。

MetaCognition 工作流 Step 1: 团队级 Wiki 共享

MetaCognition 的 Wiki 目录 (.cursor/insights/wiki/) 可以纳入版本控制 (Git LFS 或独立仓库)。

开发者 A 记录陷阱：

```
## Wiki Node: docker_volume_mount_permission_trap.md

**Symptom**: Docker 容器内无法写入挂载的卷，提示 Permission denied
**Root Cause**: 容器内用户 (uid=1000) 与宿主机目录所有者 (uid=0/root) 不一致
**Resolution**: 使用 Docker user namespace 或挂载时指定 uid/gid
**Hit Count**: 1 (by Alice)
**Tags**: #docker #volume #permission #devops
```

Step 2: 跨会话复用

开发者 B 在 2 周后遇到同样问题：

- Librarian 检索命中 docker_volume_mount_permission_trap
- **自动提示**：“Alice 在 2 周前遇到过同样的问题，解决方案是...”
- B 直接应用已知方案，**无需排查**

Step 3: 知识复利

开发者 C 在 1 个月后再遇到：

- Hit Count 达到 3 (Alice:1, Bob:1, Carol:1)
- 系统自动触发 promote_live_constraints
- 生成 Live Constraint: docker_volume_mount_uid_check
- 回写到团队共享的 CLAUDE.md

Step 4: 新人 Onboarding

新成员 Dave 加入团队，第一次构建 Docker 环境：

- 运行 docker-compose up 时，Librarian 预检命中 docker_volume_mount_uid_check
- **自动拦截**：“检测到 Docker 卷挂载，团队历史上 3 次踩坑，建议检查 uid 一致性”
- Dave 在**第一次尝试**即应用正确方案，**零踩坑**

团队价值：

- **知识传承**：从” 口头传授” 升级为” 自动感知”

- 新人效率: onboarding 时间从 2 周 (踩坑学习) 缩短到 2 天 (自动避坑)
- 文档活跃: Wiki 由系统自动维护, 不会腐烂

2.7 场景七: 新人 Onboarding

场景描述 新成员加入项目, 需要理解代码库结构、开发规范、常见陷阱。传统方式是阅读 README、Wiki、询问老员工, 但信息分散、过时、不完整。

MetaCognition 工作流 Step 1: Session Start — 知识预加载

新成员第一次启动 Claude Code:

- Librarian 自动检索项目级 Wiki
- 加载 top-10 high-severity 负向约束
- 加载项目架构决策 (Architecture Decisions)

Claude 主动提示:

欢迎来到项目! 我已加载团队的知识库, 以下是该项目最重要的 5 条防御红线:

1.  **** 禁止在 bash 中使用换行符连接命令 **** (历史上 5 次踩坑)
→ 使用 ``&&`` 替代
2.  **** 任何文件上传端点必须验证 MIME 类型 + 使用 UUID 存储 **** (历史上 5 次安全漏洞)
3.  **** 重构 parser.py:parse_config 前必须先分析调用链 **** (历史上 3 次测试雪崩)
4.  ****React 异步测试优先使用 MSW 替代 jest.mock**** (团队最佳实践)
5.  **** 性能优化前先 EXPLAIN ANALYZE, 禁止直接加缓存 **** (常见反模式)

你可以随时问我关于这些规则的详细背景, 或说 " 记住了 " 让我记录新的经验。

Step 2: 开发中的实时守护

新成员写代码时:

- 每次 tool call 前, Librarian <200ms 预检
- 命中约束时即时提示: “你正在修改的文件历史上曾导致..., 建议...”
- 不中断心流: Micro 任务零感知, Light 任务选择性提示, Full 任务才强制拦截

Step 3: 踩坑后的快速学习

新成员遇到新问题 (未在 Wiki 中):

- Debug Gate 触发, 外脑会诊解决
- 解决后自动记录为 draft insight
- 新成员的经验立即成为团队资产

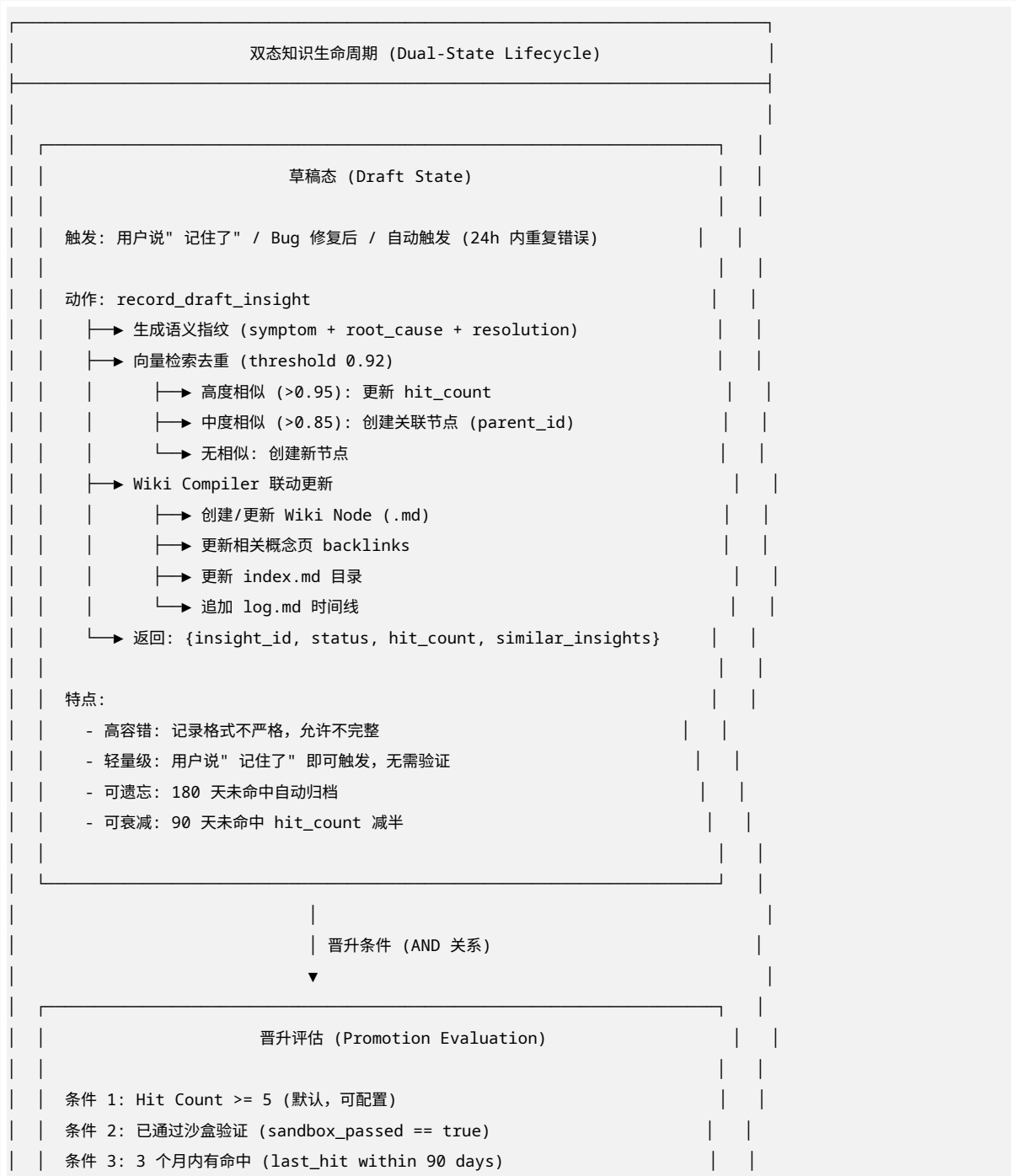
Step 4: 知识晋升

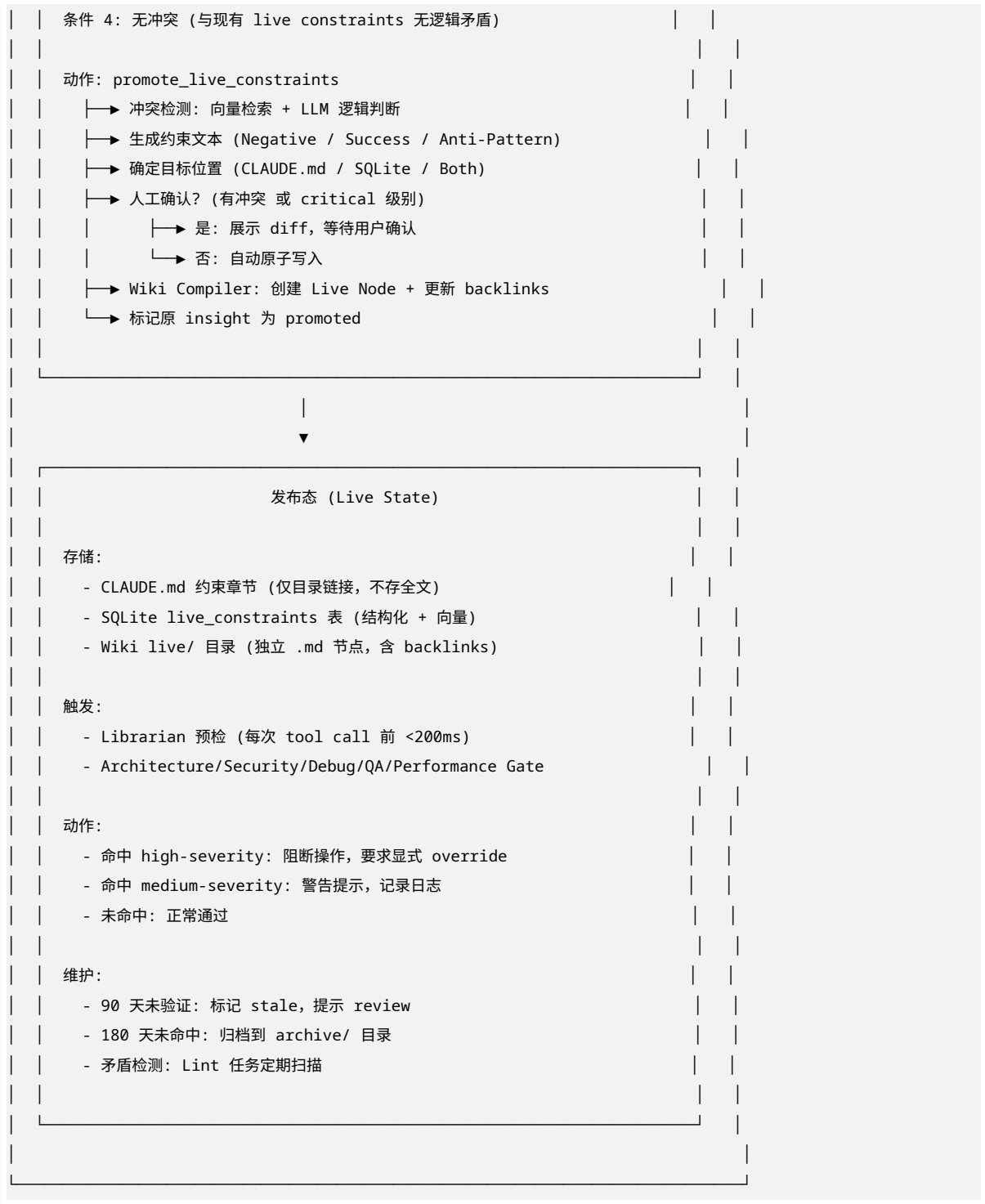
该洞察被团队其他成员复用，Hit Count 增长：

- 达到 5 时自动晋升 Live Constraint
- 所有团队成员的 CLAUDE.md 自动更新
- 知识从个人经验进化为团队规范

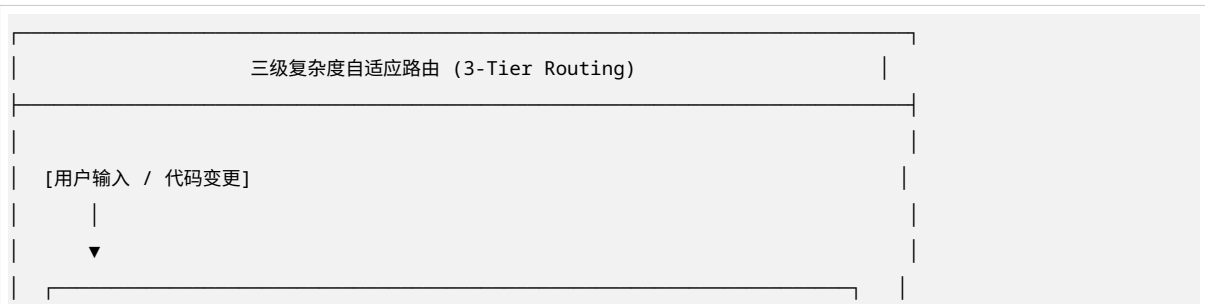
3. 业务逻辑

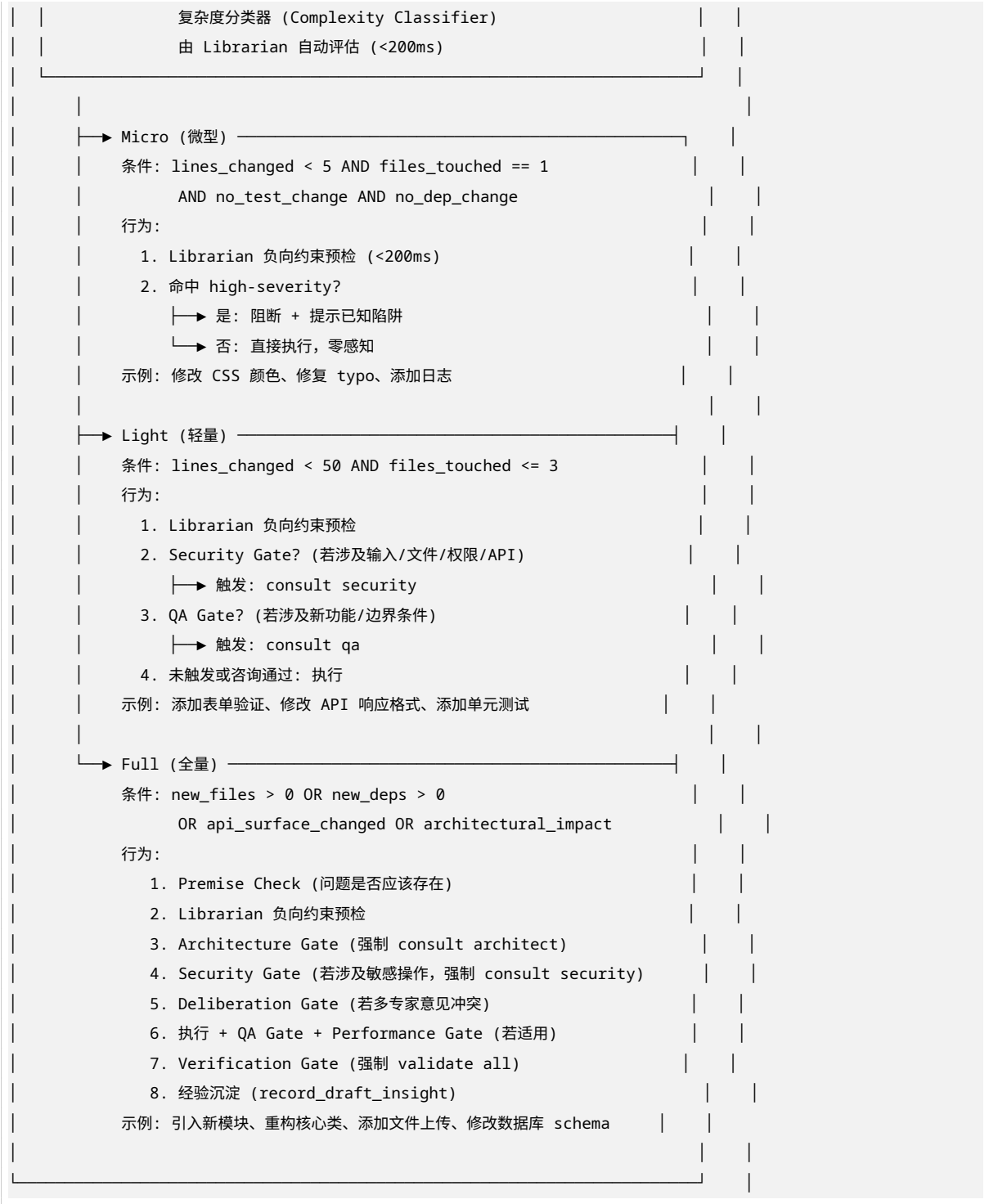
3.1 双态知识生命周期





3.2 三级复杂度自适应路由





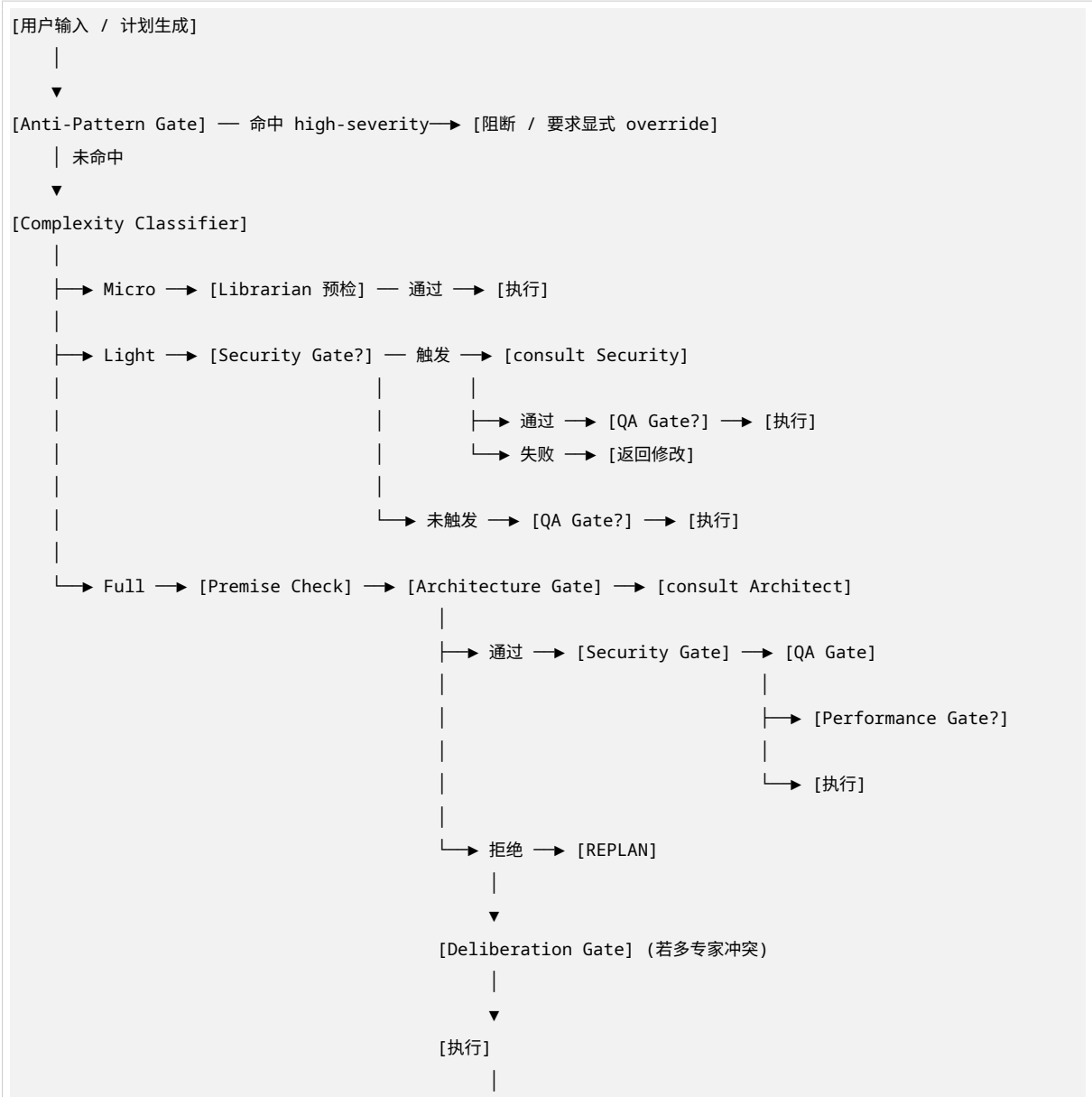
3.3 六重决策门

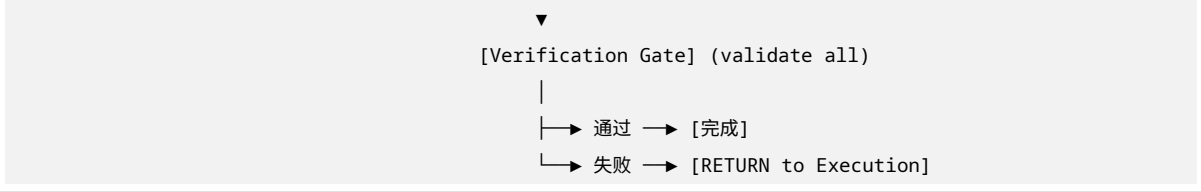
| 决策门 | 触发条件 | 咨询专家 | 复杂度 | 是否可绕过 | 失败行为 |
|-------------------|----------------------|-----------|--------------------|-------|---------------------|
| Anti-Pattern Gate | 每次 tool call 前自动拦截 | Librarian | Micro/ Light/ Full | ❌ 否 | 阻断操作, 要求显式 override |
| Architecture Gate | 新增模块、重大重构、跨服务调用、引入依赖 | Architect | Full | ❌ 否 | 回流 Planning, 重新分解任务 |

Table 2 – continued

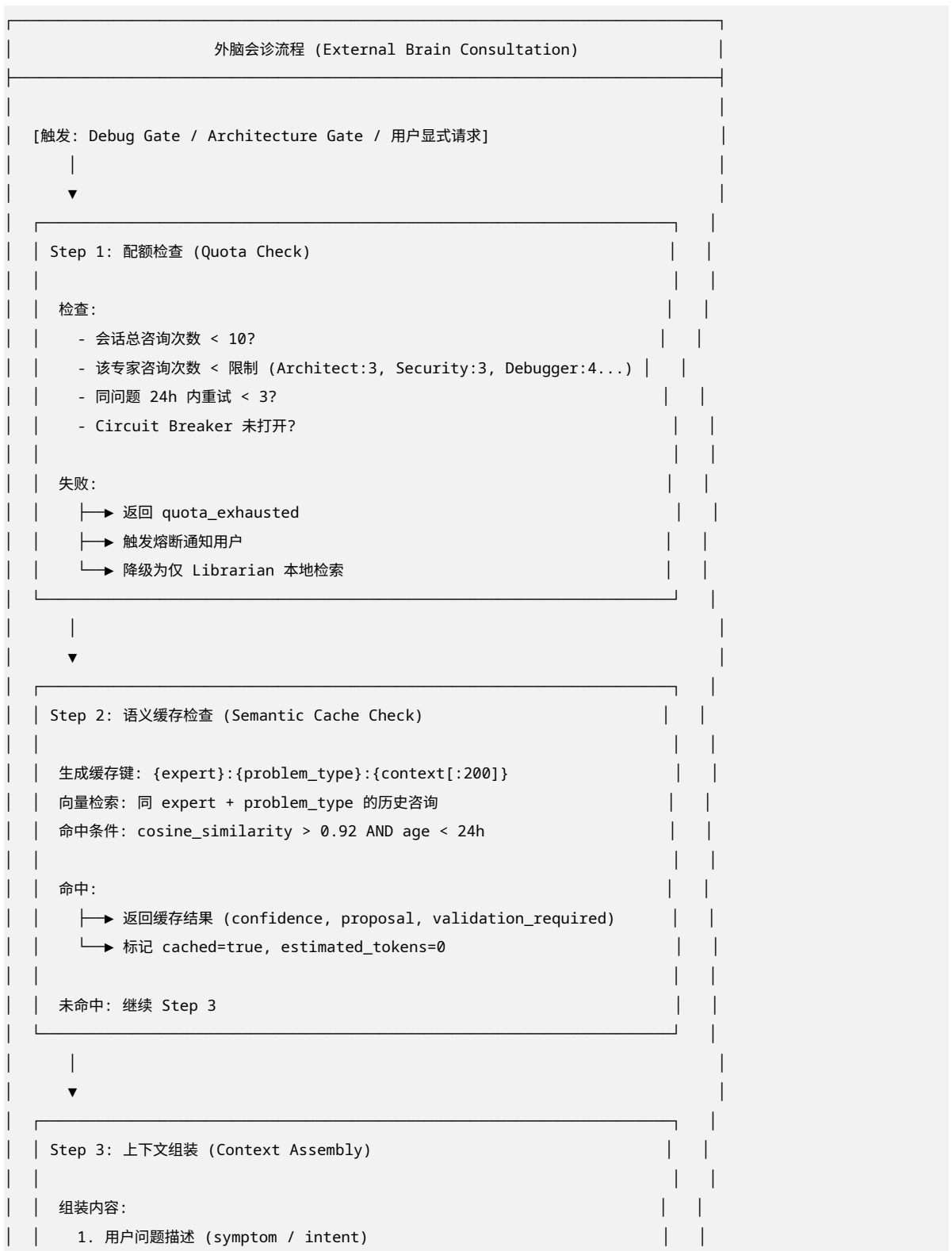
| 决策门 | 触发条件 | 咨询专家 | 复杂度 | 是否可绕过 | 失败行为 |
|------------------|-------------------------------|-------------|------------------|--------|------------------|
| Security Gate | 用户输入处理、文件上传、权限变更、外部 API 调用 | Security | Full (Light 若触发) | ❌ 否 | 阻断，必须修复安全漏洞 |
| Debug Gate | 同一错误修复 ≥ 2 次失败、未知异常、性能衰退 | Debugger | Full | ❌ 否 | 外脑会诊，禁止第 3 次盲目重试 |
| QA Gate | 新功能开发、复杂边界条件、历史 bug 区域 | QA | Light/ Full | ⚠️ 选择性 | 提示测试策略建议 |
| Performance Gate | DB 查询修改、循环嵌套、缓存策略调整 | Performance | Light/ Full | ⚠️ 选择性 | 提示性能基准要求 |

决策门状态机:





3.4 外脑会诊流程



| | | |
|---|--|--|
| 2. 相关代码片段 (affected files) | | |
| 3. 错误日志 / 堆栈跟踪 (error logs) | | |
| 4. 已尝试方案 (attempted fixes) | | |
| 5. 历史约束检索结果 (Librarian retrieve) | | |
| 6. CodeGraph Impact Graph (物理拓扑, 若适用) | | |
| 7. 项目上下文 (project_context) | | |
| 示例 (Architecture Gate): | | |
| " 请评估以下重构方案的安全性: | | |
| 代码片段: [src/parser.py:45-67] | | |
| 影响分析: 直接调用者 3 个 (main.py:load_config, cli.py:init, | | |
| api.py:parse_request), 间接影响 8 个测试文件 | | |
| 历史约束: [[parser_refactoring_fragility]] (Hit: 3) | | |
| 未测试调用者: api.py:parse_request (风险最高) | | |
| 目标: 将 parse_config 的副作用提取到接口中" | | |
| ▼ | | |
| Step 4: 模型路由 (Model Routing) | | |
| 解析配置层级: | | |
| 调用参数 model_override > 角色配置 > 全局 default | | |
| 示例路由: | | |
| Architect → Claude-Opus (复杂架构) | | |
| Security → Claude-Opus + 规则引擎 (双路审查) | | |
| Debugger → DeepSeek-R1 (深度推理, timeout 60s) | | |
| QA → GPT-4.1 (测试生成) | | |
| Performance → Gemini-2.5-pro (长上下文分析) | | |
| Librarian → Ollama Llama3.2:3b (本地轻量, <200ms) | | |
| ▼ | | |
| Step 5: 外脑调用 (External Brain Invocation) | | |
| 构建系统提示 (System Prompt): | | |
| "You are an [expert_role] consultant. | | |
| Your response must: | | |
| 1. Fit in a single implementation task of <=5 minutes | | |
| 2. Include exact file paths and verification steps | | |
| 3. If the solution is larger, decompose and return ONLY the | | |
| first task | | |
| 4. Never hallucinate APIs or methods; verify existence first" | | |
| 执行调用: | | |
| - Anthropic: Messages API (支持 Advisor Tool 快速通道) | | |
| - OpenAI: Chat Completions API | | |
| - DeepSeek: 兼容 OpenAI 协议, reasoning_effort=high | | |
| - Google: Gemini API, 1M+ token 上下文 | | |

| | | | |
|--|--|--|--|
| | - Ollama: 本地 HTTP API, 零成本 | | |
| | | | |
| | 错误处理: | | |
| | - 网络超时: 重试 2 次, 然后切换备用模型 | | |
| | - API 错误: 记录 failed_consultation, 返回降级建议 | | |

| | | |
|--|--|--|
| 错误处理: | | |
| - 网络超时: 重试 2 次, 然后切换备用模型 | | |
| - API 错误: 记录 failed_consultation, 返回降级建议 | | |

- | | | | |
|--|--|--|--|
| | - 网络超时: 重试 2 次, 然后切换备用模型 | | |
| | - API 错误: 记录 failed_consultation, 返回降级建议 | | |

| | | | |
|--|--|--|--|
| | Step 6: 响应解析与缓存 (Response Parsing & Caching) | | |
| | | | |

```
| | 解析结构化输出: | |
| | - proposal: 可执行建议 (字符串) | |
```

- | | |
|------------------------|---------------|
| - proposal: | 可执行建议 (字符串) |
| - confidence: | 置信度 (0-1) |
| - validation_required: | 是否需要沙盒验证 (布尔) |
| - estimated_tokens: | 预估 Token 消耗 |

| | | | |
|--|------------------------|--|--|
| | 写入语义缓存: | | |
| | - 向量嵌入 (context[:500]) | | |
| | - 24h 时效 | | |

- | | | | | |
|--|--|------------------------|--|--|
| | | - 向量嵌入 (context[:500]) | | |
| | | - 24h 时效 | | |

| | | |
|-------------------------------------|--|--|
| 记录咨询历史: | | |
| - 写入 consultation_history (审计 + 熔断) | | |

- | | | | |
|--|-------------------------------------|--|--|
| | - 写入 consultation_history (审计 + 熔断) | | |
|--|-------------------------------------|--|--|

| | | | |
|--|-------------------------------|--|--|
| | Step 7: 结果路由 (Result Routing) | | |
| | | | |

```
| confidence >= 0.75 AND validation_required == false: | |
```

```
| | |
```

```
| | └─ 直接应用提案，展示给用户确认 | |
```

```
| | |
```

└─> 直接应用提案，展示给用户确认

```
| confidence >= 0.75 AND validation_required == true: | |
```

```
| | |
```

```
| | |   └─ 进入 Validation Pipeline (Step 8) | |
```

```
| | | |
```

```
| | └─▶ 进入 Validation Pipeline (Step 8) | |
```

- confidence < 0.5:
 - 标记不确定性, 展示原始提案 + 免责声明
 - 建议用户补充上下文或切换方案

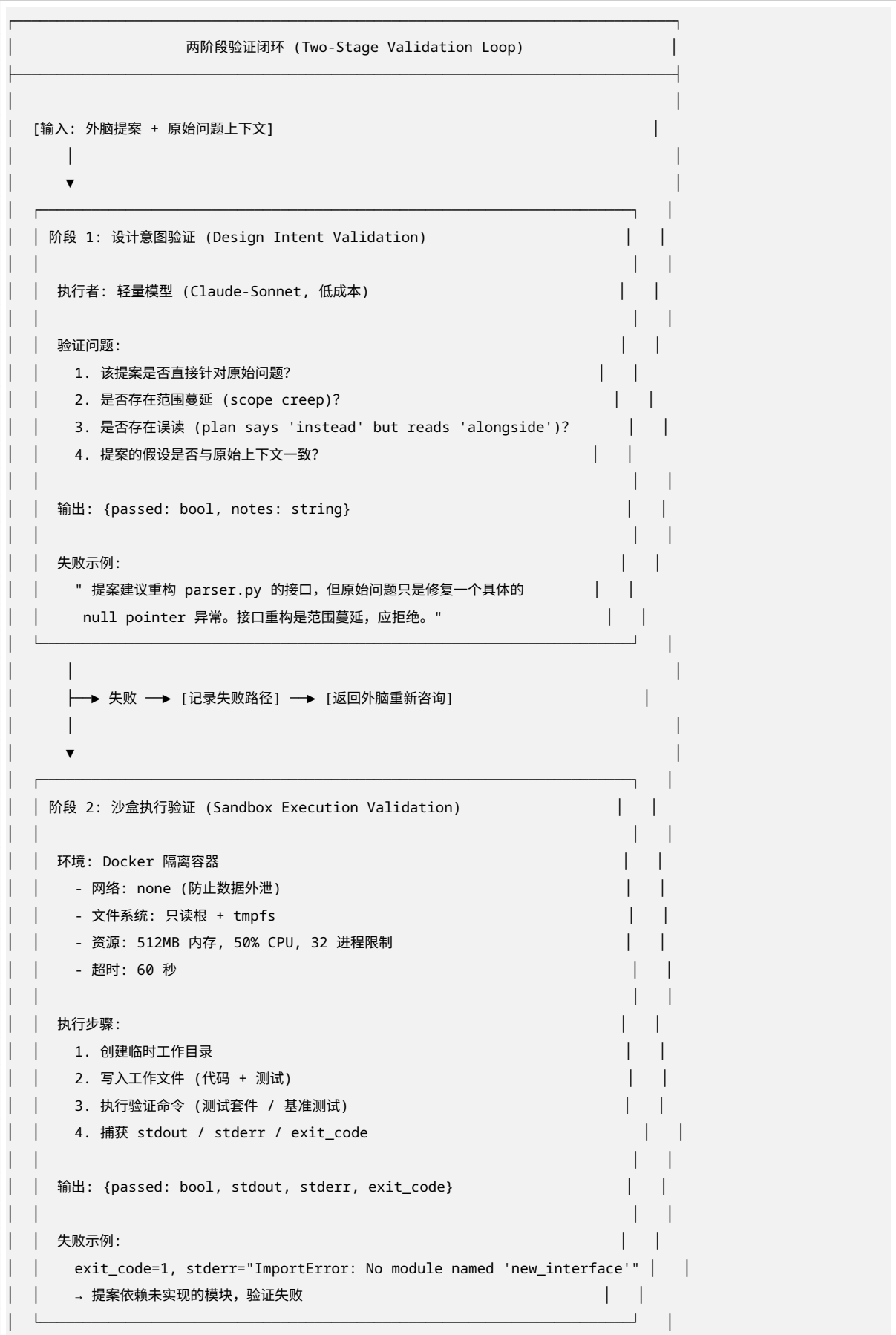
- 标记不确定性, 展示原始提案 + 免责声明
- 建议用户补充上下文或切换专家

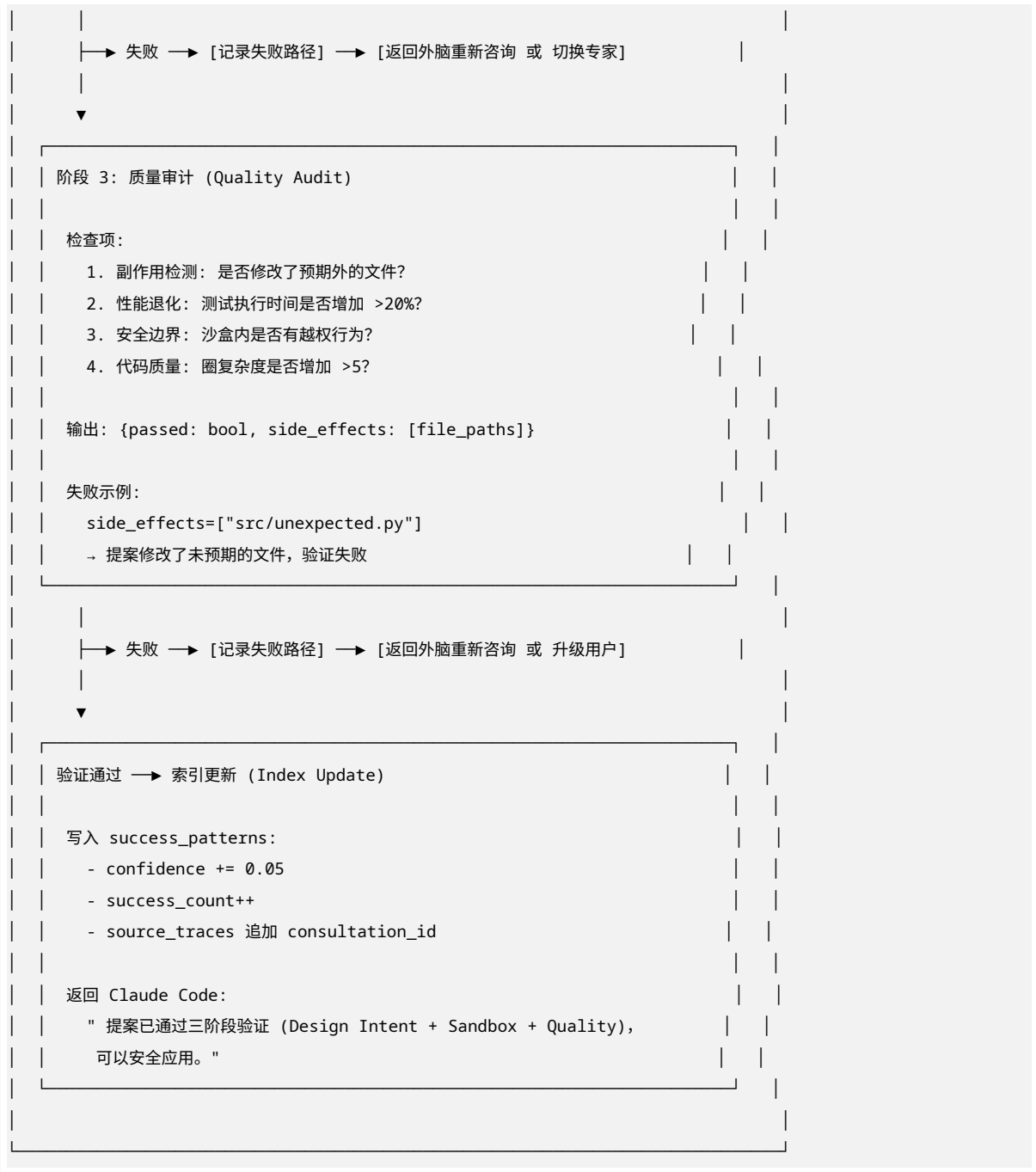
└─▶ 建议用户补充上下文或切换专家

| | | | | | |
|--|--|-------------------------------------|--|--|--|
| | | Step 8: 验证流水线 (Validation Pipeline) | | | |
| | | | | | |

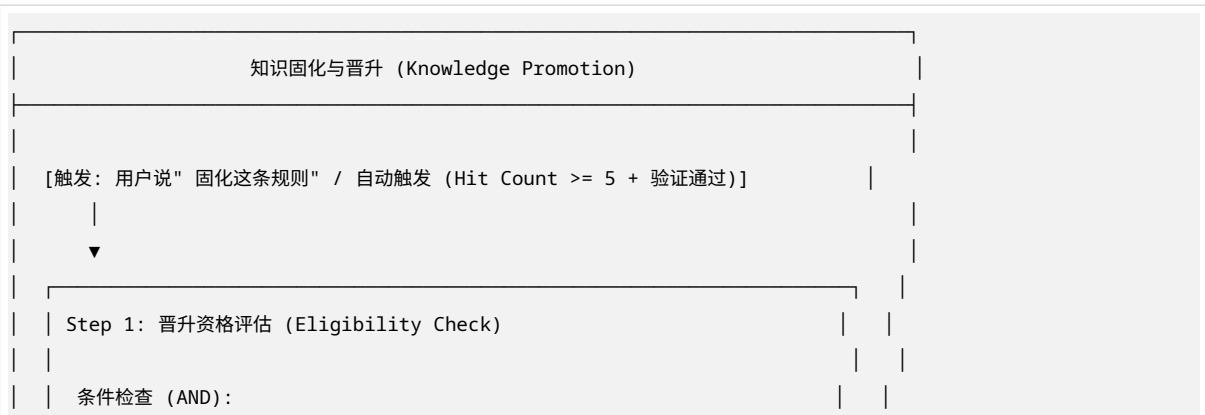
| | | | | | |
|--|--|---------|--|--|--|
| | | 见 3.5 节 | | | |
|--|--|---------|--|--|--|

3.5 两阶段验证闭环





3.6 知识固化与晋升



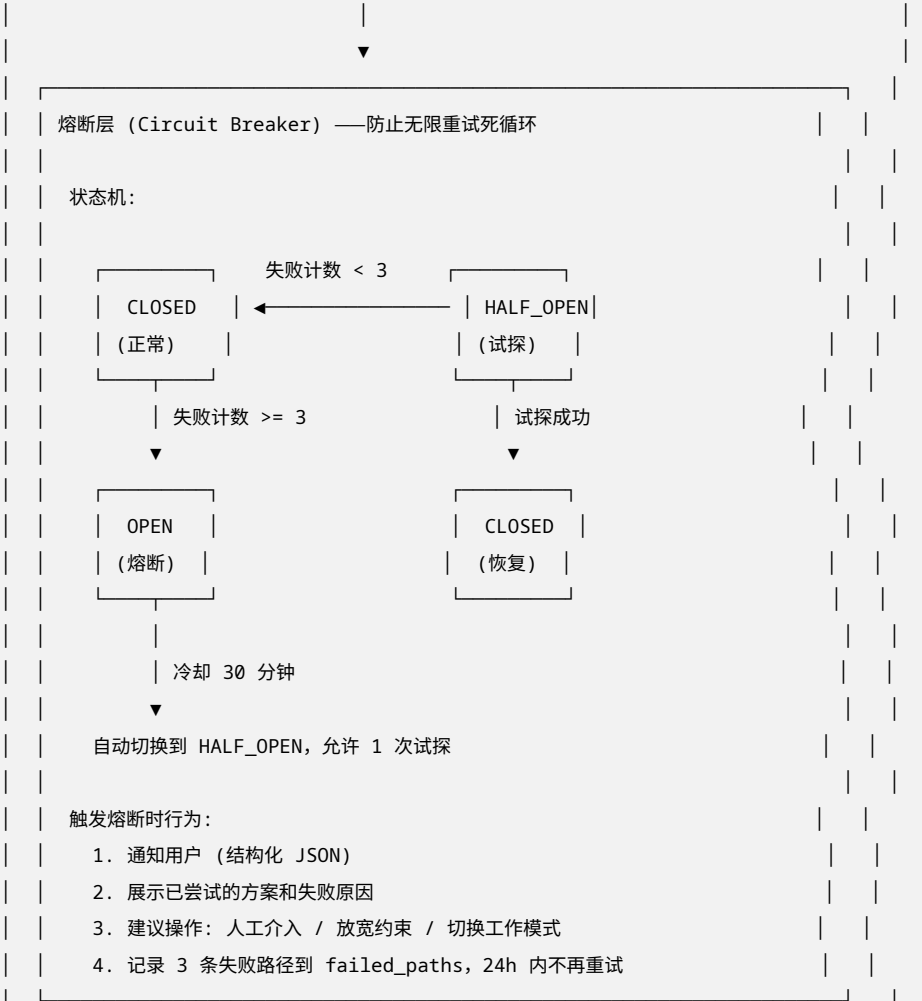
| | | |
|--|--|--|
| 1. Hit Count >= threshold (默认 5, 可配置) | | |
| 2. 验证证据: sandbox_passed == true | | |
| 3. 时效性: last_hit 在 90 天内 | | |
| 4. 状态: status == 'draft' (未已晋升) | | |
| 失败返回: | | |
| - threshold_not_met: " 当前 Hit Count 3 < 要求 5" | | |
| - insufficient_evidence: " 未通过沙盒验证" | | |
| - stale_insight: "90 天内无命中, 建议重新验证" | | |
| ▼ | | |
| Step 2: 冲突检测 (Conflict Detection) | | |
| 检索相似的 live_constraints (向量相似度 > 0.80): | | |
| - 检查是否逻辑矛盾 (需要 LLM 判断) | | |
| 示例冲突: | | |
| 新洞察: " 禁止在单元测试中直接操作数据库" | | |
| 现有约束: " 集成测试必须连接真实数据库" | | |
| → 矛盾! 需要人工裁决或细化上下文 (单元测试 vs 集成测试) | | |
| → 发现冲突 → [标记 requires_human_confirm=true] | | |
| [展示冲突约束 + diff 预览] | | |
| [等待用户确认或手动解决] | | |
| ▼ | | |
| Step 3: 约束文本生成 (Constraint Generation) | | |
| 类型选择: | | |
| - negative_constraint: " 禁止/永远不要..." | | |
| - success_pattern: " 推荐/使用... 模式" | | |
| - anti_pattern: " 避免/不要... 反模式" | | |
| 文本凝练: | | |
| 原始洞察: " 在 Windows + VS Code + Conda 环境下, bash 脚本中
conda activate 失败, 因为默认 Terminal Profile 是
PowerShell, 未执行 Conda Hook" | | |
| 凝练约束: "Windows VS Code 中使用 bash + conda 时,
必须将 Terminal Profile 设为 Git Bash" | | |
| ▼ | | |
| Step 4: 原子写入 (Atomic Write) | | |
| 目标位置: | | |
| - CLAUDE.md: 仅更新目录链接, 不追加全文 | | |

| | | |
|--|--|--|
| | - SQLite: live_constraints 表 (结构化 + 向量) | |
| | - Wiki: live/ 目录 (独立 .md 节点) | |
| | | |
| | CLAUDE.md 更新示例: | |
| | ## Development Constraints | |
| | - [[conda_path_mismatch_in_vscode_terminal]] — Hit: 5 | |
| | (Windows VS Code bash + conda 必须使用 Git Bash) | |
| | | |
| | 原子操作: | |
| | 1. 创建临时文件 (tempfile) | |
| | 2. 写入新内容 | |
| | 3. shutil.move 替换原文件 (原子操作) | |
| | 4. 失败时回滚 | |
| | | |
| | ▼ | |
| | Step 5: Wiki Compiler 联动更新 | |
| | | |
| | 创建 Live Node: | |
| | .cursor/insights/wiki/live/conda_path_mismatch_in_vscode_terminal.md | |
| | | |
| | 联动更新: | |
| | - 更新概念页: python_environment_setup.md (追加 backlink) | |
| | - 更新概念页: vscode_terminal_profiles.md (追加 backlink) | |
| | - 更新 index.md: 在 " 环境配置 " 分类下新增条目 | |
| | - 追加 log.md: "2026-05-20: Promoted conda_path_mismatch (Hit: 5)" | |
| | | |
| | 标记原 insight: | |
| | - status: 'promoted' | |
| | - promoted_to: 'live_0e41b9' | |
| | | |
| | ▼ | |
| | 完成 → 返回: {promoted_id, target_location, conflicts, requires_human_confirm, diff_preview} | |
| | | |

3.7 熔断与配额管理

| | | |
|--|-----------------------------------|--|
| | 熔断与配额管理 (Circuit Breaker & Quota) | |
| | | |
| | 配额层 (Quota Layer) ——防止 Token 恶性狂刷 | |
| | | |
| | 会话级限制: | |

- 总会话咨询次数: <= 10
 - 单专家咨询次数:
 - Architect: <= 3
 - Security: <= 3
 - Debugger: <= 4
 - QA: <= 2
 - Performance: <= 2
 - Deliberation: <= 2
- 问题级限制:
- 同 query_signature 24h 内重试: <= 3
 - 同专家连续失败: >= 2 时自动切换专家
- Token 预算:
- 默认每次咨询: <= 50K tokens
 - Debugger (推理链长): <= 100K tokens
- 耗尽行为:
- 返回 quota_exhausted
 - 降级为仅 Librarian 本地检索
 - 通知用户: " 外脑配额已耗尽, 建议: 1) 等待 24h 重置 2) 检查循环触发"



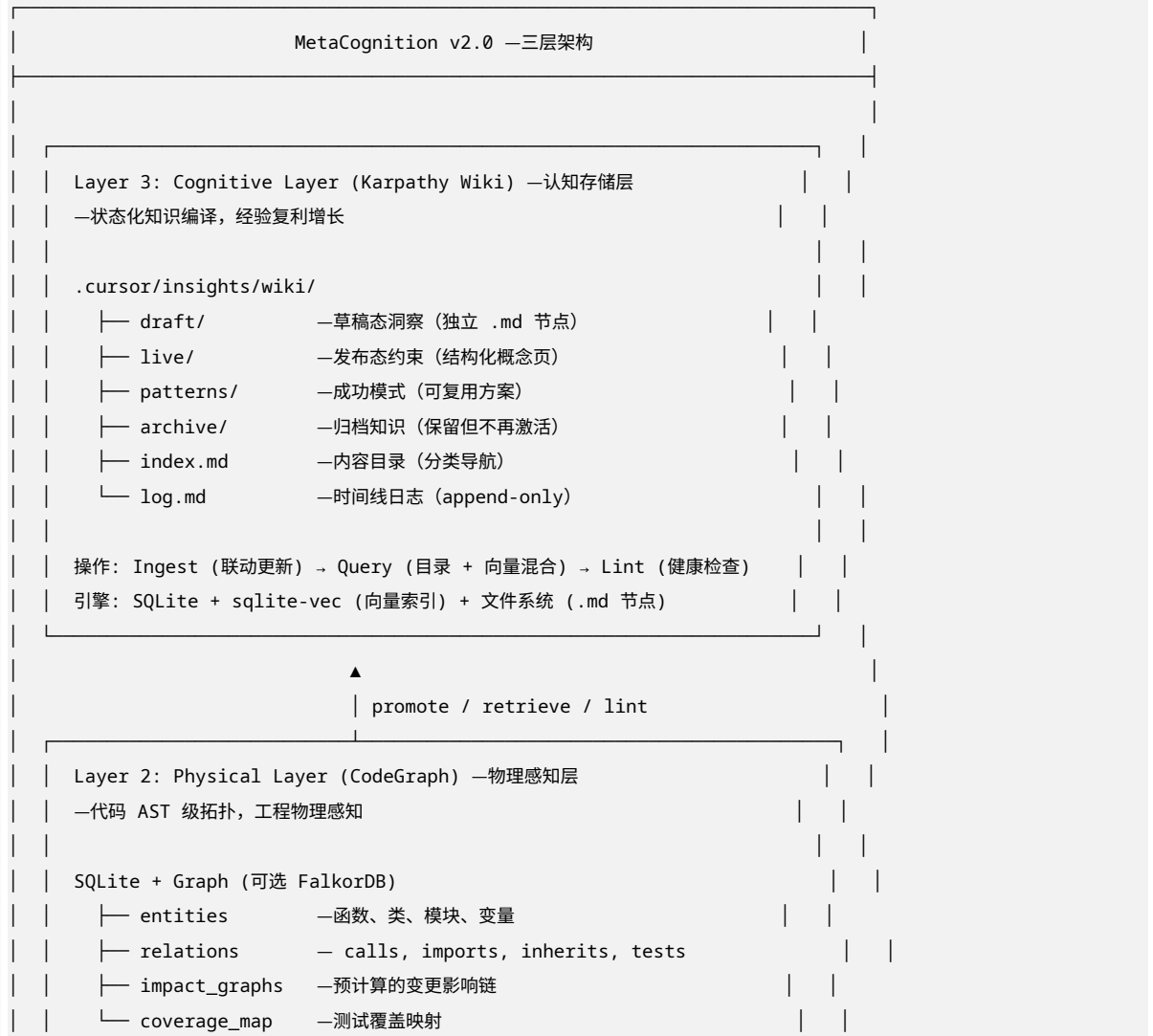
3.8 知识衰减与归档

| 知识衰减与归档 (Knowledge Decay & Archive) | | |
|--|--|--|
| 问题: 如果知识只增不减, Wiki 会无限膨胀, 旧规则会污染新上下文 | | |
| 解决方案: 三级衰减机制 | | |
| 第一级: Hit Count 衰减 (90 天周期) | | |
| 规则: 每 90 天未命中, hit_count 减半 (最低 1) | | |
| 示例: | | |
| 初始: Hit Count = 5 | | |
| 90 天后未命中: Hit Count = 2 (5 // 2) | | |
| 180 天后未命中: Hit Count = 1 (2 // 2) | | |
| 270 天后未命中: Hit Count = 1 (最低) | | |
| 效果: 偶发问题自然降级, 不再阻碍日常开发 | | |
| ▼ | | |
| 第二级: 状态归档 (180 天周期) | | |
| 规则: 180 天未命中的 draft insight, 自动标记为 archived | | |
| 行为: | | |
| - 从 active 检索中排除 | | |
| - 移动到 archive/ 目录 | | |
| - 保留在数据库中 (可手动恢复) | | |
| 示例: | | |
| "Next.js 13 的 app router 问题" ——项目已升级到 Next.js 15 | | |
| → 180 天后自动归档, 不再干扰 Next.js 15 开发 | | |
| ▼ | | |
| 第三级: 约束过时标记 (90 天周期) | | |
| 规则: live constraint 90 天未验证, 标记为 stale | | |
| 行为: | | |
| - 降级为 medium severity (不再自动阻断) | | |
| - 提示用户: " 该约束 90 天未触发, 是否仍有效? " | | |

| | | | |
|--|---------------------------------------|--|--|
| | - 用户确认有效：重置计时器 | | |
| | - 用户确认无效：标记为 deprecated，保留但不再触发 | | |
| | | | |
| | 示例： | | |
| | " 禁止在 bash 中使用换行符" ——项目已迁移到 Python 脚本 | | |
| | → 用户确认无效 → 标记 deprecated → 不再干扰 | | |
| | | | |
| | 维护任务： | | |
| | - 每日：自动衰减 Hit Count | | |
| | - 每周：自动归档陈旧 insights | | |
| | - 每月：自动标记 stale constraints | | |
| | - 每季度：Schema 共同演进（调整阈值、触发条件） | | |

4. 架构设计

4.1 三层认知架构





4.2 认知层 (Karpathy Wiki)

借鉴 Karpathy LLM Wiki 的”编译式知识管理”模式:

核心原则:

- **Raw Sources → Wiki Nodes → Compiled Knowledge:** 原始经验被 LLM 自动编译为结构化、交叉链接的知识节点

- **Ingest 联动更新**: 单个洞察可能触发 10-15 个 Wiki 页面的联动更新（创建节点、更新 backlinks、更新目录、追加日志）
- **Index + Log 双导航**: index.md 提供内容目录（按类别组织），log.md 提供时间线（append-only，可 grep）
- **Lint 健康检查**: 定期扫描矛盾、过时、孤立节点

Wiki Node 结构:

```
<!-- .cursor/insights/wiki/live/dependency_injection_gateway.md -->
# Dependency Injection: Gateway Pattern

## Constraint (Negative)
禁止在业务代码中直接引入 `llm_gateway` 依赖。任何外部交互必须通过扁平化的 `llm_client` 实施路由。

## Rationale
- 直接依赖 gateway 会导致测试时无法 mock，引入网络抖动
- 扁平化 client 支持运行时切换 provider (Anthropic/OpenAI/DeepSeek)
- 历史踩坑: [[draft_20260310_gateway_mock_failure]]

## Success Pattern (Positive)
```python
✅ 正确
from llm_client import LLMClient
client = LLMClient(provider="anthropic")

❌ 错误
from llm_gateway import Gateway
```

Backlinks

- [[test\_mocking\_strategies]] — 如何 mock llm\_client
- [[provider\_switching]] — 运行时 provider 切换模式
- [[draft\_20260310\_gateway\_mock\_failure]] — 原始踩坑记录

Compliance Check

- 触发文件: src/services/\*.py
- 检测正则: from llm\_gateway import
- 自动拦截: 是 (high severity)
- 历史命中: 5 次

Metadata

- created: 2026-03-15
- last\_validated: 2026-05-20
- version: 2
- deprecated: false

```
4.3 物理层 (CodeGraph)

借鉴 CodeGraph 的 AST 级代码拓扑感知:
```

```

** 核心能力 **:
- ** 实体提取 **: 函数、类、模块、变量 (tree-sitter 解析)
- ** 关系建模 **: calls, imports, inherits, tests
- ** 影响分析 **: 修改函数 X 会影响哪些调用者、哪些测试
- ** 覆盖映射 **: 哪些调用者有测试覆盖, 哪些没有

** 与 MetaCognition 的集成 **:

```yaml
# enriched context_fingerprint
context_fingerprint:
  git_diff_hash: "a3f2..."
  affected_files: ["src/parser.py", "tests/test_parser.py"]

code_graph:
  modified_entities:
    - type: "function"
      name: "parse_config"
      file: "src/parser.py"
      lines: [45, 67]

  upstream_callers:
    - type: "function"
      name: "load_config"
      file: "src/main.py"
      distance: 1
      has_test: true

    - type: "function"
      name: "parse_request"
      file: "src/api.py"
      distance: 1
      has_test: false # 风险点

  downstream_callees:
    - type: "function"
      name: "validate_json"
      file: "src/utils.py"
      distance: 1

  test_coverage:
    direct: 2/3 # 直接调用者中有测试的比例
    indirect: 5/8 # 间接影响中有测试的比例

risk_score: 0.7 # 基于未测试调用者 + 跨模块边数计算

```

4.4 推理层 (Expert Brain MCP)

基于 Model Context Protocol (MCP) 的多角色外脑会诊系统:

MCP Tools:

Tool	职责	输入	输出
expert_brain/ draft_insight	记录草稿洞察	symptom, root_cause, resolution, context_fingerprint	insight_id, status, hit_count
expert_brain/consult	外脑会诊	expert, context, problem_type, model_override	consultation_id, proposal, confidence, validation_required
expert_brain/validate	验证提案	consultation_id, proposal, validation_type	validation_id, passed, stage_results
expert_brain/promote	晋升约束	insight_id, promotion_type, validation_evidence	promoted_id, target_location, conflicts
expert_brain/retrieve	检索知识	query, index_type, top_k, min_confidence	按类型分组的结果列表
expert_brain/lint	健康检查	—	矛盾列表、过时节点、孤立节点
codegraph/ analyze_impact	影响分析	entity, change_type	impact_graph, risk_score
codegraph/trace_calls	调用链追踪	from, to, max_depth	call_chain
codegraph/get_coverage	覆盖查询	entity	coverage_report
codegraph/detect_cycle	循环检测	module	cycle_report

5. 功能规格

5.1 MetaCognition Skills

Skill 1: record_draft_insight 触发条件:

- 用户显式说: “记住了”、“这是个陷阱”、“以后别踩这个坑”
- Bug 修复后, 用户确认: “解决了, 记下来”
- 自动触发: 同一类错误在 24h 内第二次出现且被修复

行为契约:

```
## Skill: Record Draft Insight

**Trigger**: User explicitly signals a lesson learned, or auto-trigger on repeated bug pattern.

**Pre-condition**: The bug has been resolved and the fix is verified.

**Action**:
1. Synthesize a concise insight:
  - Symptom: What went wrong (1 sentence)
  - Root Cause: Technical explanation (1-2 sentences)
  - Resolution: Verified fix or workaround (1 sentence)
  - Context Fingerprint: Affected files + git diff hash
2. Call MCP Tool: `expert_brain/draft_insight`
3. Present result to user:
```


- If "new": "Insight recorded. Hit count: 1"
- If "updated": "Insight reinforced. Hit count: N"
- If "duplicate": "This pattern is already tracked (ID: xxx)."

****Post-condition**:** Insight exists in Draft state.

****Constraints**:**

- NEVER write to local files directly.
- NEVER append to CLAUDE.md.

Skill 2: consult_external_brain 触发条件:

- Auto (Micro): Librarian 预检命中 high-severity 负向约束 → 阻断
- Auto (Full): 同一错误修复 ≥ 2 次失败; 或新增模块/依赖/API
- Manual: 用户说“问问架构师”、“帮我审查安全”、“这个设计拿不准”

行为契约:

Skill: Consult External Brain

****Trigger**:** Auto-gate failure, user request, or deliberation gate.

****Pre-condition**:** Problem context is assembled.

****Action**:**

1. Classify the problem domain:
 - architecture → architect
 - security / auth / input → security
 - debugging / test failure / error log → debugger
 - testing / coverage / boundary cases → qa
 - performance / bottleneck / optimization → performance
 - multi-domain conflict → deliberation
2. Call MCP Tool: ``expert_brain/consult``
3. Receive structured proposal:
 - If confidence ≥ 0.75 and validation_required == false:
 - Apply proposal directly
 - If validation_required == true:
 - Proceed to ``expert_brain/validate``
 - If confidence < 0.5 :
 - Flag uncertainty

****Constraints**:**

- NEVER spawn subprocesses.
- NEVER construct raw API requests.

Skill 3: promote_live_constraints 触发条件:

- 用户显式说: “固化这条规则”、“把这个写进规范”
- 自动触发: insight hit_count ≥ 5 AND 通过沙盒验证 AND 无冲突

行为契约:

```

## Skill: Promote Live Constraints

**Trigger**: User explicit command, or auto-trigger on mature validated insight.

**Pre-condition**: Target insight exists in Draft state with sufficient hit count and validation
↳ evidence.

**Action**:
1. Call MCP Tool: `expert_brain/promote`
2. Receive promotion result:
  - If "promoted":
    → Present diff preview to user
    → If requires_human_confirm == true: wait for user "yes"
  - If "conflict":
    → Present conflicting rules, ask user to resolve
  - If "insufficient_evidence":
    → Explain what's missing
  - If "threshold_not_met":
    → Inform user of current hit count

**Constraints**:
- NEVER directly modify CLAUDE.md.
- NEVER execute SQL INSERT statements.

```

5.2 Expert Brain MCP Server

Tool: expert_brain/draft_insight

```

{
  "name": "expert_brain/draft_insight",
  "description": "Record a draft insight from a resolved bug or lesson learned",
  "inputSchema": {
    "type": "object",
    "properties": {
      "symptom": {"type": "string", "description": "User-facing description of what went wrong"},
      "root_cause": {"type": "string", "description": "Technical explanation of why it happened"},
      "resolution": {"type": "string", "description": "Verified fix or workaround"},
      "context_fingerprint": {"type": "string", "description": "Auto-generated: affected files + git
↳ diff hash"},
      "severity": {"type": "string", "enum": ["low", "medium", "high", "critical"], "default": "medium"},
      "tags": {"type": "array", "items": {"type": "string"}, "description": "Optional domain tags"}
    },
    "required": ["symptom", "root_cause", "resolution", "context_fingerprint"]
  }
}

```

Return Schema:

```

{
  "insight_id": "string",
  "status": "enum: [new, updated, duplicate]",

```

```
"hit_count": "integer",
"similar_insights": ["array of insight IDs"]
}
```

Tool: expert_brain/consult

```
{
  "name": "expert_brain/consult",
  "description": "Consult an expert for a specific domain problem",
  "inputSchema": {
    "type": "object",
    "properties": {
      "expert": {"type": "string", "enum": ["architect", "security", "debugger", "qa", "performance",
        ↵ "librarian", "deliberation"]},
      "context": {"type": "string", "description": "Full problem description"},
      "problem_type": {"type": "string", "description": "Classification for query signature"},
      "model_override": {
        "type": "object",
        "properties": {
          "provider": {"type": "string"},
          "model": {"type": "string"},
          "base_url": {"type": "string"},
          "api_key": {"type": "string"}
        }
      },
      "bypass_cache": {"type": "boolean", "default": false}
    },
    "required": ["expert", "context", "problem_type"]
  }
}
```

Return Schema:

```
{
  "consultation_id": "string",
  "proposal": "string",
  "confidence": "number 0-1",
  "validation_required": "boolean",
  "estimated_tokens": "number",
  "cached": "boolean"
}
```

5.3 与 Superpowers 融合

Superpowers Phase	MetaCognition 嵌入点	动作
Deliberation	brainstorming 产出多方案后	consult deliberation —冲突方案综合 审议

Table 4 – continued

Superpowers Phase	MetaCognition 嵌入点	动作
Planning	每个跨模块 task 后	consult architect —架构边界审查
Execution	执行前/中/后	Librarian 预检 → Debug Gate → Security Gate QA Gate → Architecture Gate → record_draft_insight → Performance Gate
Quality (TDD)	Red/Green/Refactor 每个转换点	validate(all) —Design Intent + Sandbox + Quality (强制) Librarian 检索 约束 + 自动晋升 hit_count ≥ 5 的洞察
Verification	完成声明前	
Context	Session start / 定期维护	

5.4 TDD 深度适配

TDD 阶段	MetaCognition 触发	动作
Red (写测试)	测试构造 2 次尝试失败	consult qa —测试构造策略
Red→Green (写代码)	最小方案跨模块	consult architect —架构正确路径
Green (通过)	测试通过且原因非显而易见	record_draft_insight —记录洞察
Refactor	影响 >3 测试文件或性能退化 >20%	consult architect/ performance —安全重构策略

6. 配置与部署

6.1 完整配置示例

```
# expert-brain.yaml
version: "1.0"

default:
  provider: "anthropic"
  model: "claude-sonnet-4-20250514"
  base_url: ""
  api_key: ""
  api_key_env: "ANTHROPIC_API_KEY"
  temperature: 0.2
  max_tokens: 800
  timeout_ms: 30000
  max_retries: 2

experts:
  architect:
    name: "Architect"
    model:
      provider: "anthropic"
      model: "claude-opus-4-20260201"
      base_url: ""
      api_key: "${ANTHROPIC_OPUS_KEY}"
      temperature: 0.3
      max_tokens: 1200
      advisor_tool:
        enabled: true
        max_uses: 3
    triggers: ["new_module", "major_refactor", "cross_service_call", "dependency_addition"]

  security:
    name: "Security"
    model:
      provider: "anthropic"
      model: "claude-opus-4-20260201"
      base_url: ""
      api_key: "${ANTHROPIC_SECURITY_KEY}"
      temperature: 0.1
      max_tokens: 1000
    triggers: ["user_input_handling", "file_upload", "permission_change", "external_api_call"]

  debugger:
    name: "Debugger"
    model:
      provider: "deepseek"
      model: "deepseek-r1"
      base_url: "https://api.deepseek.com/v1"
      api_key: "${DEEPSEEK_API_KEY}"
      temperature: 0.6
```

```
    max_tokens: 2000
    reasoning_effort: "high"
    timeout_ms: 60000
    triggers: ["test_failure_repeat:2", "error_unknown", "performance_regression"]

qa:
  name: "QA"
  model:
    provider: "openai"
    model: "gpt-4.1"
    base_url: ""
    api_key: ""
    temperature: 0.3
    max_tokens: 1000
    triggers: ["new_feature", "complex_boundary_conditions", "historical_bug_area"]

performance:
  name: "Performance"
  model:
    provider: "google"
    model: "gemini-2.5-pro"
    base_url: ""
    api_key: "${GOOGLE_API_KEY}"
    temperature: 0.2
    max_tokens: 1000
    triggers: ["db_query_change", "loop_nesting", "cache_strategy_change"]

librarian:
  name: "Librarian"
  model:
    provider: "ollama"
    model: "llama3.2:3b"
    base_url: "http://localhost:11434"
    api_key: ""
    temperature: 0.0
    max_tokens: 500
  auto_trigger:
    enabled: true
    intercept_mode: "pre_action"
    check_latency_ms: 200

deliberation:
  name: "Deliberation Synthesizer"
  model:
    provider: "anthropic"
    model: "claude-opus-4-20260201"
    base_url: ""
    api_key: ""
    temperature: 0.4
    max_tokens: 1500
    triggers: ["deliberation_gate"]
```

```

quotas:
  per_session:
    total_consultations: 10
  per_expert:
    architect: 3
    security: 3
    debugger: 4
    qa: 2
    performance: 2
    deliberation: 2
  per_query_signature:
    max_retries: 3
    retry_window_hours: 24
  token_budgets:
    default_per_consultation: 50000
    debugger: 100000
  circuit_breaker:
    failure_threshold: 3
    recovery_timeout_minutes: 30

validation:
  sandbox:
    type: "docker"
    image: "expert-brain-sandbox:latest"
    timeout_seconds: 60
    network: "none"
  auto_index_update:
    on_success: true
    on_failure: true

promotion:
  hit_count_threshold: 5
  stale_days: 90
  archive_days: 180
  hit_count_decay_interval: 90
  decay_factor: 0.5

escalation:
  notify_user_on:
    - "exhausted_retries"
    - "circuit_breaker_open"
    - "budget_exceeded"
    - "expert_disagreement_high"

```

6.2 Claude Desktop 配置

```

{
  "mcpServers": {
    "expert-brain": {
      "command": "python",

```

```
"args": ["/path/to/expert-brain-server/main.py"],
"env": {
  "EXPERT_BRAIN_CONFIG": "/path/to/expert-brain.yaml",
  "ANTHROPIC_API_KEY": "sk-ant-...",
  "ANTHROPIC_OPUS_KEY": "sk-ant-...",
  "ANTHROPIC_SECURITY_KEY": "sk-ant-...",
  "OPENAI_API_KEY": "sk-...",
  "DEEPSEEK_API_KEY": "sk-...",
  "GOOGLE_API_KEY": "...",
  "DATABASE_PATH": "/path/to/expert-brain.db"
}
}
```

6.3 启动时校验

Server 启动时执行以下校验，任一失败则拒绝启动：

1. **配置文件格式校验**：YAML 语法正确
2. **API KEY 明文检测**：禁止硬编码，只接受 `${ENV_VAR}` 语法
3. **环境变量可解析性**：所有引用的环境变量必须存在
4. **数据库连接**：SQLite 可读写
5. **Docker 可用性**：如果启用沙盒，docker 命令必须可用
6. **CodeGraph 解析器**：tree-sitter 语言支持检查（可选）

7. 实施路线图

Phase 1: 立法规制 (Week 1-2)

1. 创建 CLAUDE.md MetaCognition Router
2. 配置 Claude Desktop 接入 Expert Brain MCP Server
3. 提取当前最严重的 5-10 个错误模式，转化为 MUST/NEVER 规则
4. 配置 Librarian 自动拦截（Ollama 本地部署）

Phase 2: Wiki 知识库 (Week 3-4)

1. 创建 .cursor/insights/wiki/ 目录结构
2. 迁移现有约束为 Wiki Node（独立.md 文件）
3. 实现 Ingest 联动更新（创建节点 + 更新 backlinks + 更新 index + 追加 log）
4. 部署 expert_brain/draft_insight 和 expert_brain/retrieve

Phase 3: 外脑会诊 (Week 5-6)

1. 部署 expert_brain/consult，配置多提供商
2. 部署 expert_brain/validate，配置 Docker 沙盒
3. 建立两阶段验证流水线（Design Intent + Sandbox + Quality）
4. 实现熔断与配额管理

Phase 4: 闭环自动化 (Week 7-8)

- 1. 部署 expert_brain/promote, 实现自动晋升
- 2. 配置 Hit Count 衰减与归档任务
- 3. 实现 Wiki Lint 健康检查
- 4. 与 Superpowers 工作流全面融合

Phase 5: 物理感知 (Week 9-10)

- 1. 集成 CodeGraph (tree-sitter 解析)
- 2. Enrich context_fingerprint 为 Impact Graph
- 3. Librarian 物理级拦截 (循环依赖、覆盖率、复杂度)
- 4. Architecture Gate 量化增强

Phase 6: TDD 深度适配 (Week 11-12)

- 1. 在 TDD Red/Green/Refactor 各阶段植入决策门
- 2. 建立测试构造知识库 (test-construction patterns)
- 3. 建立重构安全规则库 (refactor safety constraints)
- 4. 团队级 Wiki 共享配置

8. 附录

8.1 术语表

术语	定义
MetaCognition	主智能体对自身认知过程的管理：感知无知、求助外脑、固化经验
Expert Brain	基于 MCP 的外脑服务层，封装多模型路由、缓存、验证、索引
Draft Insight	草稿态知识，高容错、轻量级捕获，未经严格验证
Live Constraint	发布态规则，经严格验证后固化到 CLAUDE.md 或索引
Decision Gate	强制咨询点，不可绕过
Query Signature	问题哈希标识，用于重试计数和熔断
Negative Constraint	“绝对不能做”的规则
Success Pattern	验证通过的解决方案模板
Anti-Pattern	已证伪的尝试方案
Two-Stage Validation	设计意图验证 + 沙盒执行验证 + 质量审计
Deliberation	多专家并行咨询 + 综合审议
Advisor Tool	Anthropic server-side 模型协作原语
Circuit Breaker	连续失败后熔断，防止资源浪费
Semantic Cache	基于向量相似度的缓存，替代 MD5 字符串匹配
Context Fingerprint	代码上下文哈希 (git diff + 受影响文件)，用于溯源
Hit Count Decay	长期未命中的洞察自动降低权重，防止僵尸规则
Wiki Compiler	自动维护 Wiki 节点的联动更新引擎
Impact Graph	基于 AST 的代码变更影响链 (调用者、被调用者、测试覆盖)
GraphRAG	图遍历 + 向量相似度融合的混合检索

8.2 参考文献

- Anthropic. *Claude Code Auto-Memory & Advisor Tool*. 2026.
- obra. *Superpowers Framework*. GitHub, ~150K stars.
- gstack. *Role-Based Decision Gates for AI Agents*. 2026.
- Karpathy, A. *llm-wiki.md: A Personal Wiki Maintained by LLMs*. Gist, 2026. cite web__search:37#7
- Model Context Protocol (MCP) Specification. *Protocol 2025-11*.
- DreamOS. *Biologically-Inspired Memory Architecture for LLM Agents*. EvolvingAgents Labs.
- /wizard Skill. *Pre-Action Checklist for Claude Code*. Dev.to, 2026.
- Self-Correction Loop. *Systematic Debugging & Memory Audit for .NET Agents*. 2026.
- dead-letter-oracle. *Governed MCP Agent with Closed-Loop Reasoning*. 2026.
- claude-concilium. *Multi-Agent AI Consultation Framework via MCP*. 2026.
- CodeGraph. *VS Code Extension for Code Visualization*. Microsoft.
- FalkorDB. *Graph Database for Code Analysis*. 2026.
- codegraph-cli. *AST-Based Code Graph Extraction Tool*. GitHub.

文档版本: 2.0

最后更新: 2026-05-20

作者: AI Architecture Team

兼容: Claude Code Desktop 0.9+, MCP Protocol 2025-11, Superpowers 2026